

MicroC/OS-II용 디바이스 드라이버 작성 ②

매트릭스 키패드 디바이스 드라이버 작성

지난 호에 이어 이번에는 임베디드 시스템의 매트릭스 키패드 드라이버 작성에 대해 살펴보자.

대형 임베디드 시스템일 경우는 여전히 PC와 같은 외장 키보드를 사용하지만, 대부분의 경우는 원가 문제나

시스템의 크기 제약 때문에 키 처리를 위한 별도의 하드웨어를 달기 어렵다. 또 PC처럼 복잡한 키를 처리할 일도 별로 없다.

간단한 하드웨어와 소프트웨어로 구현한 매트릭스 키패드는 그런 경우에 적용할 수 있는 아주 훌륭한 솔루션이다.

글: 성원호/디오이즈 대표이사

whsung@dioiz.com

임베디드 시스템에서 흔히 사용되는 매트릭스 키패드 드라이버 작성에 대해 알아본다. 키패드는 임베디드 시스템에서 사용자의 요청을 받아들이는 아주 중요한 수단이다. PC의 경우는 PS2나 USB 포트에 키보드를 연결한다. PC용 키보드는 키 처리 전용 컨트롤러를 내장하고 있기 때문에 PC에 장착된 프로세서에 아무런 부하를 주지 않고 키 입력을 처리해서 최종 키 값을 얻어낸다. PC의 키보드 하드웨어는 데이터가 들어올 때, 프로세서에게 인터럽트를 걸어주고 이때 아주 잠깐 동안만 프로세서가 키 값으로 들어온 데이터를 처리한다. 즉, 최종 키 값을 받을 때만 PC의 프로세서에 부하가 걸린다. 그 전에 일어나는 모든 처리는 키보드에 내장된 전용 컨트롤러가 담당한다.

하지만 일반적인 임베디드 시스템에서는 얘기가 다르다. 대형 임베디드 시스템일 경우는 여전히 PC와 같은 외장 키보드를 사용하지만, 대부분의 경우는 원가 문제나 시스템의 크기 제약 때문에 키 처리를 위한 별도의 하드웨어를 달기 어렵다. 또 PC처럼 복잡한 키를 처리할 일도 별로 없다. 간단한 하드웨어와 소프트웨어로 구현한 매트릭스 키패드는 그런 경우에 적용할 수 있는 아주 훌륭한 솔루션이다.

매트릭스 키패드 드라이버

전자레인지, 팩스, 복사기, 레이저 프린터, POS(Point of Sale) 터미널, 그리고 프로그램 가능한 논리 제어기(Programmable Logic Controls, PLC) 등과 같은 많은 종류의 임베디드 제품은 사용자 입력을 위해서 키보드나 키패드 인터페이스를 사용한다. 키보드는 사용자로부터 수치 데이터를 입력받는 것은 물론, 제어장치의 동작모드를 선택하는 용도로도 사용된다.

임베디드 시스템 디자이너는 제품의 원가를 항상 염두에 두어야 한다. 키보드 스캐닝을 수행하는 여러 가지 칩이 제품화돼 있기는 하지만, 소프트웨어로 구현한 키보드 스캐닝 방법은 시스템의 원가를 줄여줄 수 있을 뿐 아니라, 매우 작은 CPU 오버헤드만으로 잘 동작한다. 본격적인 설명에 앞서 마이크로프로세서가 키보드를 스캔하는 방식을 먼저 설명한다. 또한 완전하면서 이식 가능(portable)한 $m \times n$ 매트릭스 키패드 디바이스 드라이버를 소개할 것이다. 이 드라이버는 최대 8×8 까지의 크기를 갖는 어떤 배열의 키보드도 스캐닝이 가능하며, 더 많은 키를 지원하도록 쉽게 수정할 수 있다. 소프트웨어로 구현한 키패드 드라

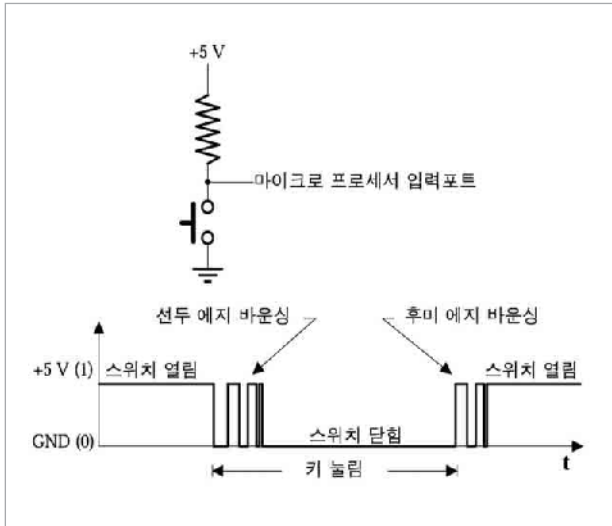


그림 1. 마이크로프로세서의 키 스위치

이버는 임베디드 시스템에 있어서 중요한 구성 요소다. 이번 호에 소개할 키패드 드라이버는 다음과 같은 특징이 있다.

- 3×3에서 8×8까지 어떤 배열의 키보드도 스캔 가능.
- 버퍼링 제공(버퍼 크기 설정 가능).
- 자동반복 기능 지원.
- 키가 얼마나 오래 눌러있는 지를 알아내는 기능.
- 쉬프트(Shift) 키를 3개까지 지원.

이 드라이버를 사용하기 위해서 해야 할 일은 하드웨어에 맞도록 3개의 하드웨어 인터페이스 함수를 수정하고 17개의 #define 상수 값을 바꿔주는 것 뿐이다. 이 키보드 드라이버는 MicroC/OS-II 상에서 동작하는 것을 가정하고 있지만, 다른 실시간 커널이나 전경/배경 환경에서 동작할 수 있도록 쉽게 수정할 수 있다.

키보드 기초

키보드의 키는 일반적으로 순간접촉 방식 스위치가 사용된다. 마이크로프로세서는 그림 1과 같은 회로를 사용해서 키 스위치의 접촉을 쉽게 감지할 수 있다. 풀 업(pull-up) 저항은 스위치가 열려 있을 때는 1의 논리를, 닫혀 있을 때는 0의 논리를 마이크로프로세서에 제공한다. 그러나 스위치가 열리고 닫히는 순간에 정확하게 1 또는 0의 논리를 제공하는 것은 아니다. 사

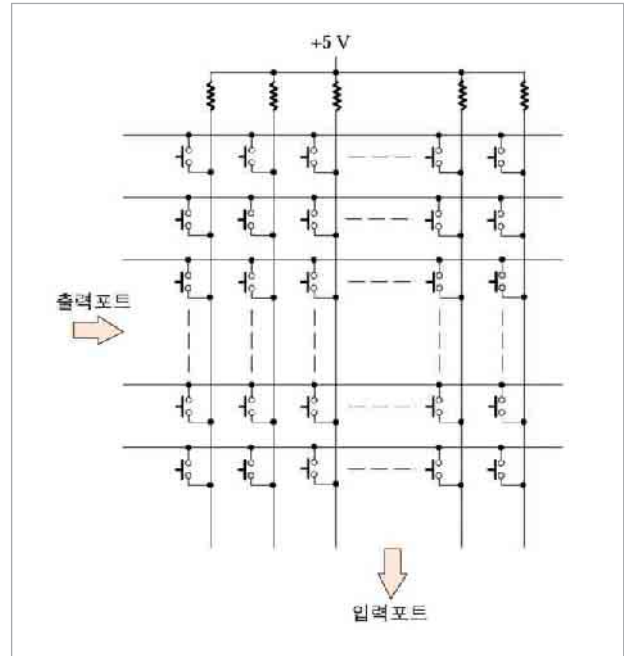


그림 2. 키보드 스위치 배치(2차원 매트릭스 형태)

람이 스위치를 닫기 위해 아무리 확실하고 빠르게 키를 누른다 해도, 이 동작은 마이크로프로세서의 입장에서 보면 상대적으로 느린 동작이다. 스위치가 닫힐 때 접촉부의 신호는 마치 공이 튀는 것처럼 동작하는데, 이런 효과 때문에 그림 1과 같은 여러 개의 펄스신호가 발생한다. 이런 현상이 일어나는 시간은 일반적으로 5~30msec 이다. 키 입력을 처리하는 소프트웨어는 이런 현상을 염두에 두고 적절한 처리를 해야 한다.

키가 여러 개 필요하다면 키로 사용할 모든 스위치를 마이크로프로세서의 각 입력 포트에 연결하면 된다. 그러나 이런 방법을 사용하면 필요한 스위치의 개수가 증가함에 따라 마이크로프로세서에 달려있는 모든 입력 포트를 순식간에 사용해 버리게 될 것이다. 키보드 상에 스위치를 배치할 때 가장 효율적인 방법은 그림 2와 같이 2차원 매트릭스 형태로 나열하는 것이다. 그 중에서도 가장 효율적인 방법은 같은 수의 행과 열을 갖는 정방 매트릭스 상에 키를 배치하는 것이다. 순간 접촉방식 스위치(누름 버튼)는 이들 행과 열의 교차점에 위치한다. 매트릭스 상에 배치할 키의 개수는 키패드를 어떤 응용에 사용할 것인가에 따라 달라진다. 각 행(row)은 출력 포트에 연결돼 있으며 마이크로프로세서가 어떤 값을 출력하는지에 따라 각 행의 논리가 변한다. 각 열(column)은 저항에 의해 풀 업(pull-up)돼 있으며,

입력 포트의 각 비트와도 연결돼 있다.

키패드 스캐닝이란 일정한 시간 간격으로 키패드 상의 키가 눌러있는 지를 조사하는 과정이다. 키패드 스캐닝 소프트웨어는 키가 눌러있다고 판단하면, 먼저 바운싱(bouncing, 키 접촉부가 공처럼 튀는 현상)을 필터링한 뒤에 실제 어떤 키가 눌러있는 지를 알아낸다. 각 키는 스캔코드(scan code)라고 하는 고유 값이 있다. 응용프로그램은 눌린 키의 스캔코드에 따라 다음 동작을 어떻게 할 지 결정한다. 즉, 응용 프로그램은 스캔코드를 보고 어떤 키가 눌렸는지 판단한다.

실수이던 고의던 간에 한 개 이상의 키를 누르는 것을 롤오버(rollover)라 한다. n-1개의 키가 눌러져 있는 상태에서 새로운 키가 하나 더 눌러지는 것을 정확하게 감지할 수 있는 알고리즘은 'n키 롤오버 기능이 있다'고 한다. 여기서 소개하는 매트릭스 키패드 디바이스 드라이버는 코드 크기를 최소화하기 위해 롤오버 알고리즘을 구현하지 않았다. 여기서 소개된 코드는 작은 임베디드 시스템을 대상으로 하기 때문에 사용자는 키를 1개씩만 누른다고 가정한다. 작은 임베디드 시스템은 일반적인 컴퓨터 시스템처럼 모든 기능을 갖춘 키패드가 필요 없다.

매트릭스 키패드 스캐닝 알고리즘

시스템 초기화시, 모든 행(출력 포트)은 0의 논리로 초기화된다(그림 2 참조). 아무 키도 눌러있지 않다면 모든 열(입력 포트)은 1의 논리로 읽힌다. 임의의 키가 눌릴 때 전체 열 중 한 열의 논리값은 0으로 바뀐다. 따라서 임의의 시각에 키 입력을 감지하려면 입력 포트를 읽어보면 된다. 입력 포트를 읽은 결과 모든 비트가 1이 아닐 경우는 키가 눌린 것이다. 일단 키 입력을 감지하면, 어떤 키가 눌렸는지 판단해야 한다. 이 과정은 상당히 간단하다.

마이크로프로세서는 전체 행 중 한 행으로만 0의 논리를, 나머지 행으로는 1의 논리를 출력한다. 이때 입력 포트 값을 읽어서 논리값이 0인 열이 발견되면 선택된 행에 있는 키 중 하나가 눌러있다고 판단할 수 있다. 반대로 입력 포트의 모든 비트가 1이라면 선택된 행에서는 눌린 키가 없다고 판단할 수 있으므로 다음 행을 선택해서 눌린 키를 찾을 때까지 앞의 동작을 반복한다. 일단 행을 찾은 뒤에는 눌린 키에 해당하는 열의 위치를 알아내고, 찾아낸 행과 열의 좌표를 이용해서 어떤 키가 눌러있는지 판단할 수 있다. 이 단계를 실행하는데 걸리는 시간은 최소 스위치 폐쇄시간에 비해 매우 짧기 때문에 그 시간 동안 스위치

는 눌러있다고 가정할 수 있다.

마이크로프로세서는 키의 바운싱 현상을 필터링하기 위해 디바운스 주기(debounce period)라고 하는 일정한 간격으로 키패드의 상태를 읽어낸다. 디바운스 주기는 사용 스위치의 특성에 따라 20ms에서 100ms 정도가 될 수 있다.

눌린 키의 스캔코드는 응용프로그램이 그 값을 읽어가기 전까지 버퍼에 저장되는 것이 일반적이다. 버퍼링은 응용프로그램이 키가 눌러질 때 곧바로 처리하지 못할 경우, 눌린 키 값을 잃어버리는 것을 방지해 주는 좋은 방법이다. 버퍼의 크기는 여러 분의 응용프로그램에 따라 다르겠지만 10개 정도의 스캔코드를 저장할 수 있을 정도면 무난할 것이다. 키 버퍼는 일반적으로 원형 큐(circular queue) 형태로 구현한다. 키가 눌리면 그 키에 해당하는 스캔코드는 큐의 다음 빈 공간에 저장된다. 응용프로그램이 키패드 드라이버로부터 스캔코드를 읽어 갈 때는 가장 먼저 저장된 스캔코드를 읽는다. 큐의 저장 공간이 꽉 찬 경우 키를 더 이상 저장할 수 없기 때문에 그 키 값은 잃어버릴 수 밖에 없다. 따라서 응용프로그램은 버퍼가 차기 전에 반드시 키 값을 읽어야 한다.

또 하나의 유용한 기능은 자동반복(auto-repeat) 또는 타이매틱(typematic)이라는 것이다. 자동반복은 사용자가 같은 키를 계속 누르고 있고 버퍼가 차지 않는 한, 눌린 키의 스캔코드를 반복적으로 버퍼에 저장하는 기능이다. 키를 반복해서 누르고 펄 필요 없이 파라미터(즉, 변수)의 값을 증가 또는 감소하고자 한다면 자동반복 기능이 그에 대한 멋진 해결책이 될 수 있다. 그림 3의 타이밍 다이어그램은 자동반복 기능이 어떻게 동작하는 지를 잘 보여준다. 키 눌림이 감지되자마자 눌린 키의 스캔코드는 버퍼에 저장된다. 키가 자동반복 시작 지연(auto-repeat start delay) 보다 오랫동안 눌러 있으면 스캔코드는 다시 버퍼에 저장된다. 그 다음부터는 키가 눌러있는 한, 스캔코드는 매 자동반복 지연(auto-repeat delay)마다 버퍼에 저장된다.

또 이 드라이버는 키가 얼마 동안 눌러있는 지 알 수 있는 기능을 제공한다. 따라서 응용프로그램은 키가 눌러있는 시간에 비례해서 파라미터의 값을 증감하는 속도를 더 빠르게 할 수 있다.

시스템의 원가를 절감하기 위해서 여러 분은 각 키에 여러 가지 기능을 할당할 수 있다.

전자계산기처럼 각 키를 다른 기능으로 사용하기 위해 프리픽스(prefix) 키를 할당할 수도 있고, 하나 이상의 쉬프트 키를 제공할 수도 있다. 프리픽스 키를 누른 후 여러 분이 원하는 키를

누름으로써 그 키에 할당된 다른 기능을 사용할 수 있다. 원래 기능을 다시 사용하려면 프리픽스 키를 다시 눌러야 한다. 쉬프트 키를 사용할 때는 먼저 쉬프트 키를 누른 상태에서 다른 키를 누르면 된다. 키패드 스캐닝 모듈은 양자 모두 키 조작을 지속적으로 감시하고 각 키에 대응하는 고유한 스캔코드를 응용프로그램에 전달한다. 여기서 소개하는 매트릭스 키패드 드라이버는 두 번째 기능만 지원하는데, 사용자는 쉬프트 키를 세 개까지 사용할 수 있다. 응용프로그램에서 눌린 키를 지속적으로 감시할 수 있다면 프리픽스 키를 직접 구현할 수도 있다.

매트릭스 키패드 드라이버

매트릭스 키패드 드라이버의 전체 소스코드 계층은 지면관계상 생략하겠다. 키패드 드라이버와 관련된 모든 함수와 변수는 Key로 시작한다. #define으로 정의된 모든 상수는 KEY_로 시작한다. 여기서는 실시간 커널 상에서 드라이버를 구현할 때 알아야 할 중요한 사항만 설명한다. 전체 소스코드에 관심이 있다면 [참고서적\[2\]](#)를 보기 바란다.

이 드라이버는 8×8 이내에서 임의의 행과 열의 크기를 갖는 모든 키패드를 스캐닝 할 수 있다. 행(row)은 8비트 단위로 출력 포트에 의해 구동된다. 이 드라이버는 행이 출력 포트의 비트 0부터 위치한다고 가정한다. 열(column)은 8비트 단위로 입력 포트에 연결돼 있다. 행과 마찬가지로 열 또한 비트 0부터 위치한다. 쉬프트 키를 사용해야 한다면 열 입력 비트 중 1비트를 쉬프트 키 입력으로 사용할 수 있다. 이 드라이버는 쉬프트 키를 3개까지 제공한다. 쉬프트 키는 입력 포트의 비트 7부터 시작해서 할당된다. 즉, 첫 번째 쉬프트 키는 비트 7에, 다음 쉬프트 키는 비트 6에, 그 다음 쉬프트 키는 비트 5에 할당된다. 리스트 1과 리스트 2에 게재한 드라이버는 그림 4와 같이 4행 6열의 일반 키와 2개의 쉬프트 키가 있는 매트릭스 키패드를 사용한다는 가정 하에 설정 및 테스트했다. 각 키는 그에 상응하는 스캔코드가 할당돼 있다(그림 4 참조). 쉬프트 키가 눌러지지 않으면 스캔코드는 0~23의 값을 갖는다. 쉬프트 1 키가 눌러지

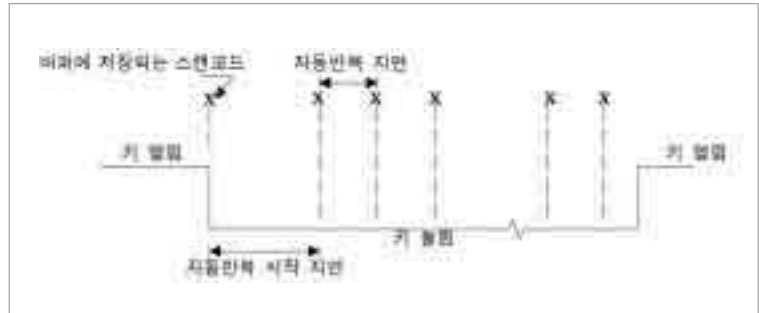


그림 3. 타이밍 다이어그램

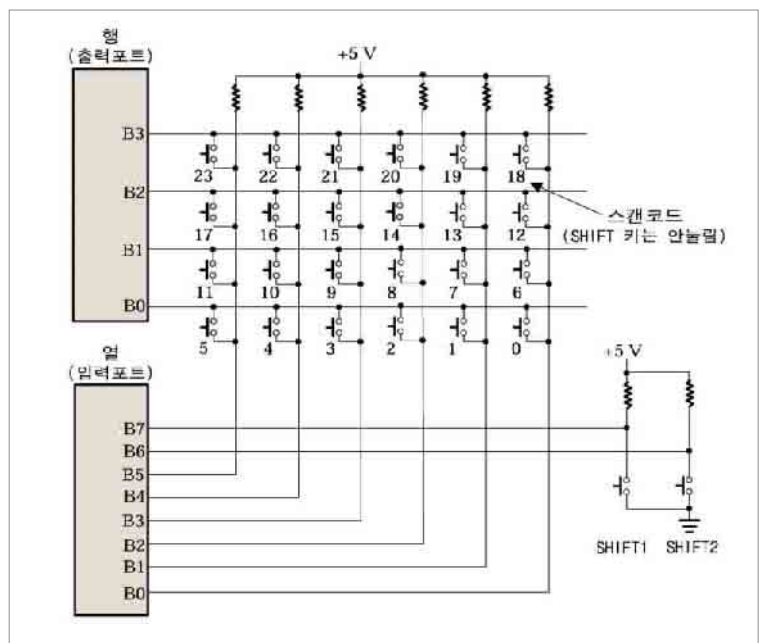


그림 4. 매트릭스 키패드 설정(할당된 스캔코드)

표 1. 키보드 스캔코드

스캔코드	눌린 쉬프트 키
0 .. 23	없음
24 .. 47	쉬프트 1
48 .. 71	쉬프트 2
72 .. 95	쉬프트 1과 쉬프트 2

면 스캔코드는 원래 값에 24를 더한 값이 된다. 이와 비슷하게 쉬프트 2 키가 눌러지면 스캔코드는 원래 값에 48을 더한 값이 된다(표 1 참조).

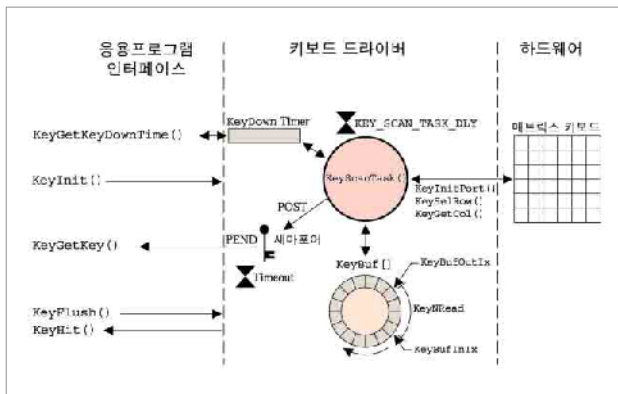


그림 5. 매트릭스 키패드 드라이버의 흐름도

내부동작

그림 5는 매트릭스 키패드 드라이버의 흐름도이다. 이 드라이버를 사용하려면 하드웨어 인터페이스 함수 3개와 17개의 #define 상수를 여러 분의 환경에 맞도록 변경해야 한다. 그림 5에 나와 있듯이 이 드라이버는 실시간 커널 상에서 동작하는 것을 가정하고 있다. 키보드 스캐닝 드라이버에서 필요한 커널 서비스는 세마포어와 시간지연 관련 함수뿐이다. 실시간 커널의 개념과 커널이 제공하는 커널 서비스에 관한 내용은 참고서적[1]을 참조하기 바란다. KeyScanTask()는 키패드 드라이버의 메인 태스크

로, 키보드 스캐닝을 담당한다. KeyScanTask()는 여러 분의 응용프로그램에서 KeyInit()을 호출할 때 생성된다. 즉, 이 드라이버를 사용하려면 응용프로그램에서 시스템 초기화시 KeyInit()을 호출해야 한다. 리스트 1은 KeyInit() 함수를 보여준다.

```
void KeyInit (void)
{
    KeySelRow(KEY_ALL_ROWS); /* 모든 행을 0으로 출력 */
    KeyScanState = KEY_STATE_UP; /* 키보드 스캐닝 태스크의 초기상태 설정 */
    KeyNRead = 0; /* 버퍼에 들어갈 키 값 개수용 변수 초기화 */
    KeyDownTmr = 0;
    KeyBufInIx = 0; /* 버퍼에 저장용 인덱스 변수 */
    KeyBufOutIx = 0; /* 버퍼 읽기용 인덱스 변수 */
    KeySemPtr = OSSemCreate(0); /* 세마포어 초기화 */
    KeyInitPort(); /* 키패드 드라이버에서 사용되는 I/O 포트 초기화 */
    OSTaskCreate(KeyScanTask, (void *)0, &KeyScanTaskStk
    [KEY_SCAN_TASK_STK_SIZE], KEY_SCAN_TASK_PRIO);
}
```

리스트 1

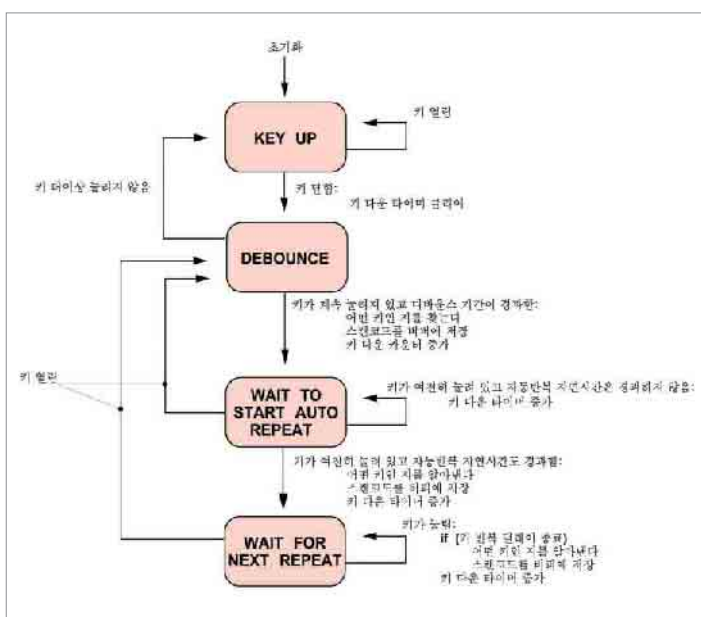


그림 6. 키패드 드라이버의 순서도

KeyScanTask()는 생성된 뒤에 KEY_SCAN_TASK_DLY 밀리초(msec)마다 깨어나서 잠깐씩 실행된다. 스캔 주기는 10에서 30Hz 사이가 되도록 KEY_SCAN_TASK_DLY를 설정하는 것이 적당하다(스캔 주기는 $1000/\text{KEY_SCAN_TASK_DLY}$ 로 결정된다).

지금까지 설명한 기능을 지원할 수 있도록 키패드를 스캔하는 가장 간단한 방법은 그림 6과 같은 상태기계(state machine)를 구성하는 것이다. 상태기계는 매 디바운스 주기마다 실행된다. 태스크는 매 KEY_SCAN_TASK_DLY 밀리초마다 실행될 때, 네 가지 상태 중 반드시 한 상태에서 실행된다.

상태기계의 초기상태는 KEY_STATE_UP 이다. 키가 눌리면 상태기계의 상태가 KEY_STATE_DEBOUNCE로 변하고 태스크는 KEY_SCAN_TASK_DLY 밀리초만큼 시간지연을 한다.

여기서 MicroC/OS-II가 제공하는 시간지연 함수인

OSTimeDlyHMSM()을 사용하면 편리하게 키 입력을 디바운스 하고, 일정간격으로 키를 스캔할 수 있다.

KeyScanTask()는 시간지연 후에도 키가 여전히 눌러있는 지를 확인하기 위해 KEY_STATE_DEBOUNCE 상태에 해당하는 코드를 실행한다. 만약 키가 눌러있지 않다면 상태기계는 KEY_STATE_UP 상태로 돌아간다. 그러나 키가 눌러있다면 KeyDecode()가 실행되어 그 키에 해당하는 스캔코드를 알아내고, 스캔코드는 KeyBufIn() 함수에 의해 원형버퍼에 저장된다. 이때 원형버퍼가 꽉 차 있다면 KeyBufIn()은 새로 획득한 스캔코드를 버린다. KeyBufIn()은 키보드 세마포어에 신호를 보내고, 여러 분의 응용프로그램은 KeyGetKey() 함수를 통해 버퍼에 저장된 스캔코드를 읽어올 수 있다. 상태기계는 그 다음 KEY_STATE_RPT_START_DLY 상태로 변하게 된다.

만약 키가 KEY_RPT_START_DLY 이상 눌러 있으면 자동반복 기능이 동작된다. 이 경우, 스캔코드가 버퍼에 저장되면 상태기계는 KEY_STATE_RPT_DLY 상태가 된다. 키가 더 이상 눌러있지 않다면 상태기계의 상태는 키가 열릴 때의 디바운싱을 처리하기 위해 KEY_STATE_DEBOUNCE로 변한다. 키가 눌릴 때와 마찬가지로 열릴 때도 디바운싱이 필요하다.

KeyScanTask()는 스캔 주기 이후에 KEY_STATE_RPT_DLY 상태에 해당하는 코드를 실행한다. 여기에서 눌러진 키의 스캔코드는 매 KEY_RPT_DLY 스캔타임 마다 버퍼에 저장된다. 다른 상태와 마찬가지로 자동반복 상태에서도 키가 열릴 때는 디바운싱 처리가 필요하기 때문에 KEY_STATE_DEBOUNCE로 변한다. 리스트 2는 상태기계로 구현된 키패드 스캐닝 태스크의 실제코드를 보여준다.

```
static void KeyScanTask (void *data)
{
    INT8U code;

    data = data; /* 컴파일러 경고를 피한다 (uC/OS-II req.) */
    for (;;) {
        OSTimeDlyHMSM(0, 0, 0, KEY_SCAN_TASK_DLY);
        /* 키보드 스캐닝 딜레이 */
        switch (KeyScanState) {
            case KEY_STATE_UP:
                /* 키가 눌러져 있는 지를 확인할 필요가 있음 */
```

```
        if (KeyIsKeyDown()) { /* 키눌림을 확인한다 */
            KeyScanState = KEY_STATE_DEBOUNCE;
            /* 디바운스 스테이트로 상태 변경 */
            KeyDownTmr = 0; /* 카다운 타이머를 리셋 */
        }
        break;

    case KEY_STATE_DEBOUNCE: /* 키 눌림, 스캔코드를 확인 */
        if (KeyIsKeyDown()) { /* 키가 눌렸는가? */
            code = KeyDecode();
            /* 눌린 키의 스캔코드를 알아낸다 */
            KeyBufIn(code); /* 스캔코드를 버퍼에 저장 */
            KeyRptStartDlyCtr = KEY_RPT_START_DLY;
            /* 자동반복 기능을 시작하기 위해 딜레이 시작 */
            KeyScanState = KEY_STATE_RPT_START_DLY;
        } else {
            KeySelRow(KEY_ALL_ROWS); /* 모든 행을 선택 */
            KeyScanState = KEY_STATE_UP;
            /* 키가 눌러져 있지 않다 */
        }
        break;

    case KEY_STATE_RPT_START_DLY:
        if (KeyIsKeyDown()) {
            /* 아직도 키가 눌러져 있는가? */
            if (KeyRptStartDlyCtr > 0) {
                /* 자동반복 기능 시작전에 딜레이가 필요한가? */
                KeyRptStartDlyCtr--;
                /* 예, 자동반복 시작을 위해 카운터를 감소 */
                if (KeyRptStartDlyCtr == 0) {
                    /* 자동반복을 위한 딜레이가 완료되었는가? */
                    code = KeyDecode();
                    /* 스캔코드를 알아낸다 */
                    KeyBufIn(code);
                    /* 버퍼에 스캔코드를 저장한다 */
                    KeyRptDlyCtr = KEY_RPT_DLY;
                    /* 다음 반복을 위한 딜레이 값을 로드 */
                    KeyScanState = KEY_STATE_RPT_DLY;
                }
            }
        } else {
            KeyScanState = KEY_STATE_DEBOUNCE;
```

```

        /* 키가 눌러져 있지 않다 */
    }
    break;

case KEY_STATE_RPT_DLY:
    if (KeyIsKeyDown()) {
        /* 아직도 키가 눌러져 있는가? */
        if (KeyRptDlyCtr > 0) { /* 키 반복전에 기다
할 필요가 있는가? */

            KeyRptDlyCtr--;
            /* 예, 감소하고 다음 키반복을 기다린다 */
            if (KeyRptDlyCtr == 0) { /* 키를 반복
할 때인가 */

                code = KeyDecode();
                /* 스캔코드를 알아낸다 */
                KeyBufIn(code);
                /* 스캔코드를 저장한다 */
                KeyRptDlyCtr = KEY_RPT_DLY;
                /* 자동반복에 필요한 딜레이 값을 리로드
한다 */

            }
        }
    } else {
        KeyScanState = KEY_STATE_DEBOUNCE;
        /* 키가 눌러져 있지 않음 */
    }
    break;
}
}
}
}

```

리스트 2

세마포어 이용

이 드라이버에서도 세마포어를 아주 적절히 사용하고 있다. 리스트 1에서 드라이버 초기화 함수인 KeyInit0을 보면 아래와 같이 키 버퍼용 세마포어를 0으로 초기화하는 것을 볼 수 있다. 이렇게 초기화하는 이유는 버퍼를 처음 초기화할 때 들어있는 키 값이 아무것도 없기 때문이다.

```
KeySemPtr = OSSemCreate(0); /* 세마포어 초기화 */
```

리스트 3은 버퍼에 저장된 키 값을 읽어갈 때 응용프로그램에서 호출하는 함수다. 버퍼에 아무 키 데이터도 없는 상태에서 임의의 태스크가 최초로 이 함수를 호출하면 세마포어의 카운트 값이 0이기 때문에 태스크는 대기상태가 된다. 이것은 아주 편리한 메커니즘이다. 버퍼에 저장된 키 값을 읽기 위해 함수를 호출했는데 아무 데이터도 없으면 호출자는 어떤 동작을 해야 할까? 실시간 커널을 사용하지 않을 경우는 지속적으로 함수를 호출해서 키 값이 들어오기를 기다려야 한다. 그러면서 쓸데없이 CPU 시간을 소모한다. 하지만 실시간 커널이 제공하는 세마포어 기능을 사용하면 키 데이터가 없을 경우 태스크를 대기상태로 만들 수 있으므로 공연히 CPU 시간을 소모하면서 데이터가 들어오기를 기다릴 필요가 없다.

```

INT8U KeyGetKey (INT16U to)
{
    INT8U code;
    INT8U err;

    OSSemPend(KeySemPtr, to, &err); /* 키가 눌러질 때를 기다린다 */
    OS_ENTER_CRITICAL(); /* 크리티컬 섹션을 시작한다. 인터럽트는 디세
이블 된다 */

    if (KeyNRead > 0) { /* 키보드 버퍼에 스캔코드가 있는 지를 확인한다 */
        KeyNRead--; /* 읽혀진 키의 개수를 감소시킨다 */
        code = KeyBuf[KeyBufOutIx];
        /* 버퍼로부터 스캔코드를 읽어온다 */
        KeyBufOutIx++;

        if (KeyBufOutIx >= KEY_BUF_SIZE) { /* 키보드 버퍼의 인덱
스를 조절한다 */
            KeyBufOutIx = 0;
        }

        OS_EXIT_CRITICAL(); /* 크리티컬 섹션을 종료한다 */
        return (code); /* 눌러진 키의 스캔코드를 리턴한다 */
    } else {

```

```

OS_EXIT_CRITICAL(); /* 크리티컬 섹션을 종료한다 */

return (0xFF); /* 버퍼에 스캔코드가 없음. -1을 리턴한다 */
}
}

```

리스트 3

리스트 4는 디바운스 필터링을 통과한 키 값을 버퍼에 저장할 때 사용하는 함수다. 이 함수는 우선 버퍼가 가득 차 있는지 점검한 뒤, 저장할 공간이 있을 경우만 키 값을 버퍼에 저장한다. 저장 이 끝나면 세마포어 카운트 값을 1 증가하기 위해 OSSemPost() 함수를 호출한다. 이때 키 값을 기다리면서 세마포어에 대기 중인 태스크들이 있다면 그 태스크들 중 우선순위가 가장 높은 태스크가 준비상태가 되고, 그 태스크의 우선순위가 키패드 스캐닝 태스크보다 높다면 바로 문맥전환이 발생한다. 문맥전환이 발생하면 새로 실행하는 태스크는 리스트 3에서 OSSemPend() 바로 밑의 코드를 실행해서 키패드 스캐닝 태스크가 새로 저장한 키 값을 읽어간다.

```

static void KeyBufIn(INT8U code)
{
    OS_ENTER_CRITICAL(); /* 크리티컬 섹션을 시작함. 인터럽트가 비활성화한다 */

    if (KeyNRead < KEY_BUF_SIZE) { /* 버퍼 오버플로우인지 확인한다 */
        KeyNRead++; /* 키 숫자를 증가한다 */
        KeyBuf[KeyBufInIdx++] = code; /* 스캔코드를 버퍼에 저장한다 */
        if (KeyBufInIdx >= KEY_BUF_SIZE) {
            /* 다음번에 저장될 버퍼 인덱스를 조절한다 */
            KeyBufInIdx = 0;
        }
        OS_EXIT_CRITICAL(); /* 크리티컬 섹션을 종료함 */
        OSSemPost(KeySemPtr);
        /* 버퍼에 스캔코드가 저장된 경우 세마포어를 시그널링함 */
    } else { /* 버퍼가 꽉 찼음. 스캔코드를 읽어버림 */
        OS_EXIT_CRITICAL(); /* 크리티컬 섹션을 종료함 */
    }
}

```

리스트 4

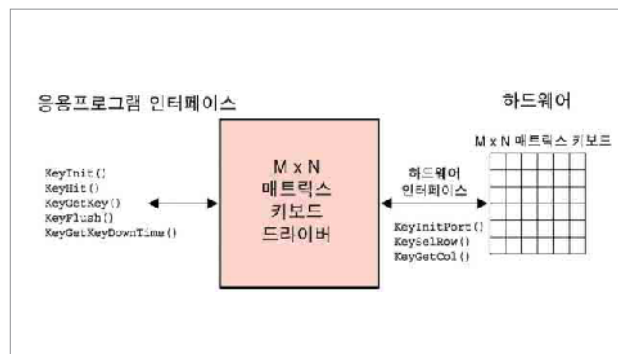


그림 7. 키패드 드라이버의 블록 다이어그램

이상으로 살펴본 바와 같이 키패드 드라이버도 세마포어를 사용해서 응용프로그램에서 편리하게 사용할 수 있는 함수를 제공한다. 여기서 사용하는 세마포어의 카운트 값은 0에서 KEY_BUF_SIZE까지 변할 수 있다. 이런 세마포어를 카운팅 세마포어라고 하며, 9월호에서 공부한 문자 LCD용 세마포어와는 차이가 있다. 문자 LCD용 세마포어의 카운트 값은 0 아니면 1이었다. 그리고 키 버퍼용 세마포어는 카운트 값을 0으로 해서 초기화하는 반면에 LCD용 세마포어는 초기 카운트 값을 1로 해서 생성한다.

인터페이스 함수

그림 7은 매트릭스 키패드 드라이버의 블록 다이어그램을 보여준다. 여러 분의 응용프로그램은 KeyFlush(), KeyGetKey(), KeyGetKeyDownTime(), KeyHit() 그리고 KeyInit() 다섯 개의 함수만을 이용해서 키보드 드라이버를 사용할 수 있다. 나머지 3개는 하드웨어에 의존적인 인터페이스 함수이며, 키패드 하드웨어를 어떻게 구현했는가에 따라 그 내용이 달라진다. 여기에 소개된 키패드 드라이버를 MicroC/OS-II가 동작하는 다른 플랫폼에서 사용해야 한다면 이 3가지 함수만 수정하면 된다.

하드웨어 인터페이스 함수 포팅

이제 원래 참고서적[2]에 소개된 키패드 드라이버를 DTB-196-BK라는 MicroC/OS-II 실습키트에서 동작하도록 포팅한다. 앞에서 말했듯이 포팅 작업은 매우 간단하다. 하드웨어 인터페이스 함수 3개와 몇 개의 #define 상수만 수정하면 된다. DTB-196-BK에는 인터폰이나 전화기에서 흔히 사용되는 3×4 키패드가 제공된다. 하드웨어 작업은 이 키패드를 80c196kc



그림 8. 키패드와 실습보드 연결

의 ioport1에 연결하는 것뿐이다. 그림 8-a는 키패드와 실습보드를 연결한 사진이다.

그림 8-b는 하드웨어 연결을 보여준다. 80c196kc에 달려있는 ioport1은 입출력을 동시에 지원하는 아주 편리한 양방향 포트다. 여기서는 ioport1의 하위 4비트를 출력으로, 상위 3비트를 입력으로 사용한다. 최상위 비트(비트7)는 사용하지 않는다.

다음은 수정해야 할 세 함수 중 하나인 열(column) 입력 포트를 읽는 함수다.

```
INT8U KeyGetCol(void)
{
    INT8U val;

    val = ioport1;
    val = ~val;
    val >>= 4;
    val &= 0x07;
    return val;
}
```

이 함수는 먼저 ioport1의 값을 읽은 뒤 그 값을 반전시킨다. 키패드의 입력 포트는 ioport1의 B6, B5, B4에 연결돼 있고 평상시에는 1의 논리값을 갖는다. 키를 누르면 해당 열의 비트 논리값은 1에서 0으로 변한다. 이 함수는 눌린 열에 해당하는 비트

의 논리값을 1로 리턴해야 하므로 실제 읽은 값을 반전한다. 다음으로 반전한 8비트 값을 오른쪽으로 4비트 이동한다. 비트 이동하면서 없어지는 하위 4비트 값은 입력이 아니라 출력이기 때문에 상관없다. 그 다음, 열에 해당하는 비트 논리값만 리턴하기 위해 0x07로 마스킹한다.

다음은 키패드와 연결된 입출력 포트를 초기화하는 함수다. 원래 책에 있는 드라이버는 8255의 입출력 포트를 초기화하는 코드가 있었지만, 실습키트에서는 ioport1에 0xf0을 써주는 것으로 초기화가 끝난다.

```
void KeyInitPort(void)
{
    ioport1 = 0xf0;
}
```

다음은 매트릭스 키패드의 행(row) 논리값을 0으로 출력하는 함수다. 어떤 행의 논리값을 0으로 출력할지는 전달인자로 알려준다. 이 함수를 호출한 뒤 상위 4비트는 1의 논리값을 유지하는 것을 주목하기 바란다. 하지만 이 함수에 의해 선택된 행 상에 있는 키를 누를 경우 해당 열의 논리 값은 0으로 바뀐다.

```
void KeySelRow(INT8U row)
{
    if (row == KEY_ALL_ROWS) {
        ioport1 = 0xf0; /* 모든 행을 0으로 출력 */
    } else {
        row = ~(1 << row);
        row |= 0xf0;
        ioport1 = row; /* 원하는 행만 0으로 출력 */
    }
}
```

다음으로 해 줄 일은 KEY.H에 있는 #define 상수를 리스트 5처럼 수정하는 것이다.

```
#define KEY_MAX_ROWS 4 /* 키보드의 최대 행(row) 수 */
#define KEY_MAX_COLS 3 /* 키보드의 최대 열(column) 수 */
```

```
#define KEY_SCAN_TASK_DLY 50 /* 키를 스캔하는 시간 간격 */
#define KEY_SCAN_TASK_PRIO 20
/* 키보드 스캐닝 태스크의 우선순위 */
#define KEY_SCAN_TASK_STK_SIZE 50
/* 키보드 스캐닝 태스크의 스택 크기 */

#define KEY_RPT_DLY 1
/* 자동 반복기능이 다시 수행되기 전의 스캔타임 값 */
#define KEY_RPT_START_DLY 10 /* 자동 반복기능이 수행되기까지의 스캔타임 값 */
```

리스트 5

먼저, 키패드 행(KEY_MAX_ROWS)과 열(KEY_MAX_COLS)의 값을 3과 4로 지정한다. 그 다음으로 스캔 주기를 50msec으로 설정한다. 키패드 스캔 태스크의 우선순위를 20으로, 이 태스크가 사용할 스택 깊이를 50으로 설정한다. 자동반복 기능으로 키 값을 저장하는 지연시간(KEY_RPT_DLY)과 자동반복 기능을 실행할 때까지 키를 누르고 있어야 할 지연시간(KEY_RPT_START_DLY)을 설정한다. 스캔 주기가 50msec이므로 결과적으로 두 지연시간은 각각 50msec와 500msec가 된다.

이것으로 간단하게 드라이버 포팅이 끝났다. 이제 응용프로그램은 초기화 함수인 KeyInit()을 호출한 뒤, KeyGetKey() 함수만 호출하면 언제든지 버퍼에 들어있는 키 값을 읽어올 수 있다. 이 드라이버가 더욱 유용한 것은 실시간 커널이 제공하는 세마포어를 사용한다는 점이다.

응용프로그램에서 키 처리 태스크를 별도로 두고, 그 태스크가 KeyGetKey() 함수를 호출해서 키를 처리하도록 하면 키 입력이 없을 때 태스크는 대기상태가 된다. 즉, 키 입력이 없을 때 키 처리 태스크가 실행을 멈추고 대기상태로 들어가기 때문에 프로세서를 점유하지 않는다. 따라서 함수를 호출했을 때 버퍼에 키 데이터가 없어서 키 데이터가 들어올 때까지 쓸데없이 루프를 반복적으로 실행해야 하는 번거로움을 없앨 수 있다. 키 값이 없어서 대기상태에 머물러 있던 태스크는 드라이버가 키 입력을 감지해서 버퍼에 키 값을 넣는 순간 깨어나서 키 입력을 처리한다. 아래 코드는 이 드라이버를 사용하는 태스크의 예를 보여준다.

```
void Task1(void *data)
{
    BYTE err, key, cnt;
    WORD i, j, k;
    cnt = key = k = 0;
    KeyInit();

    while(1)
    {
        key = KeyGetKey(0);
        key = UserKeyMapTbl[key];
        err = CommPutChar(key, 0);
        DispChar(1, 14, key);
    }
}
```

리스트 6

이 태스크는 키 드라이버를 초기화한 뒤, 무한루프를 돌면서 키 입력을 처리한다. 무한루프를 돌면서 처리하는 일은 구체적으로 살펴보면 KeyGetKey() 함수를 호출해서 드라이버가 버퍼에 저장해 놓은 스캔코드를 읽은 뒤, ASCII 코드로 변환해서 시리얼 포트와 문자 LCD로 출력하는 것이다. 앞서서도 말했듯이 KeyGetKey() 함수를 호출했을 때 버퍼에 키 값이 없을 경우, 태스크는 대기상태가 된다. 이 키패드 드라이버에서 제공하는 모든 소스코드와 함수에 대한 설명은 참고서적[2]을 보기 바란다. $R_{T_{inc}}^{\text{rel}}$

저작권 및 참고서적

이 글에 게재한 일부 내용과 소스코드는 한국내 판권자인 (주)에이콘출판사의 허가를 받아 다음 서적에서 인용했습니다.

[1] "MicroC/OS-II 실시간 커널, 제2판" (ISBN 898997533-6)

[2] "Embedded Systems Building Blocks, 2nd Edition" (ISBN 89-951540-4-7)